

Architecture Document

System Overview

The application is structured into three main parts:

- UI components (screens and controls)
- Game logic (fight system / fight tree)
- State management and navigation

The system separates the user interface from the fight logic so that gameplay behavior can be modified without changing UI components. This allows the game system to be extended (ex: adding new attacks or mechanics) without restructuring the entire app.

Component Hierarchy

The app is organized using a stack-based navigation system.

Top Level

- App
 - NavigationContainer
 - StackNavigator
 - FightSelectScreen
 - FightScreen
 - ResultScreen

FightSelectScreen

- FlatList (list of fights)
 - FightListItem
- SelectFightButton

FightScreen

- EnemySprite
- HealthBar (Player)
- HealthBar (Enemy)
- GuardButton
- DuckButton
- ExitButton

ResultScreen

- ResultText (Win / Loss)
- ReturnButton

Data Model / Fight Data

Each fight is defined using the following data set:

```
fight = {  
  id: number,  
  name: string,  
  vertices: int[][],  
  states: [  
    {  
      image: string,  
      duration: number,  
      isAttack: boolean,  
      attackType: "duck" | "guard" | "both"  
    }  
  ]  
}
```

- The **vertices graph** defines valid transitions between enemy states, as well as the likelihood of an enemy state getting picked.
- The **edges array** define basic information about each enemy state (image, timing, attack type)

Game State

```
gameState = {  
  playerHealth: number,  
  enemyHealth: number,  
  currentEdge: number,  
  isGuarding: boolean,  
  isDucking: boolean  
}
```

This state is continuously updated during gameplay and drives all UI updates.

Fight System (Core Logic)

The fight system is implemented as a form of weighted directed graph.

Each edge represents a phase for the enemy. There are three basic types of phases (idle, windup, attack) with each attack and wind having 3 different variants (duck, dodge, either). For a total of 7 phases.

- Note: Fights can be made to have more than 7 states if needed. The above is just a simple basis.

Transitions between states occur after an amount of time defined by the duration value given by the current state. The next state is determined by a system using a random number generator and the vertices graph.

- The diagram at the bottom of the page, percentages are used to show the likelihood of a state transition. This is just to give an example, and not necessarily indicative of how the actual system will work.

Core Operations

- Get current enemy state
- Get image associated with enemy state
- Check if enemy state is an attack
- Determine attack type (guard, duck, either)
- Get next enemy state
- Handle timing between enemy states
- Update current enemy state

These operations represent the logic of the fight system and are not tied to a specific implementation.

State Management Strategy

State is managed primarily within the FightScreen.

- useState is used to store:
 - player health
 - enemy health
 - current node
 - player input (guard / duck)
- useEffect is used to:
 - run the game loop
 - update fight system timing
 - transition between nodes
 - check win/loss conditions

State Flow

1. User selects a fight in FightSelectScreen
2. Fight data is passed to FightScreen
3. Fight system initializes starting enemy state
4. Game loop updates state over time
5. UI components re-render based on enemy state
6. When one of the health values reaches 0 → navigate to ResultScreen

Navigation Structure

The app uses stack-based navigation.

- FightSelectScreen → initial screen
- FightScreen → pushed when fight starts
- ResultScreen → pushed when fight ends

Navigation Flow

- Select Fight → FightScreen
- Fight Ends → ResultScreen
- Return → FightSelectScreen

External Services

The application is primarily client-side and does not require external APIs.

The system uses:

- JavaScript timers to control enemy behavior and timing
- React Native touch input for gameplay interaction
- Optional local storage (AsyncStorage) to store high scores or progress

System Connections

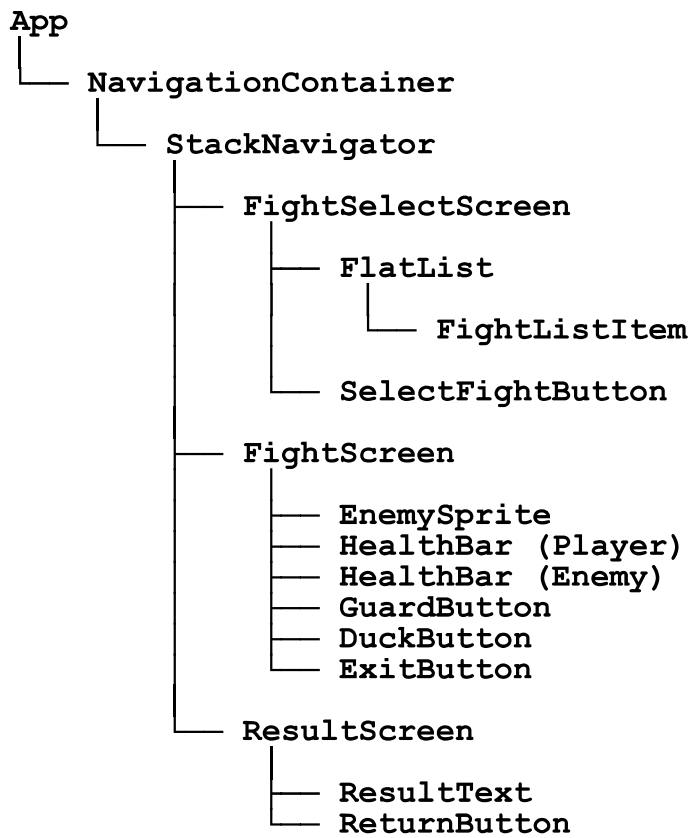
The system components interact as follows:

- FightSelectScreen passes selected fight data to FightScreen
- FightScreen initializes the fight system using that data
- The fight system updates game state (health, node, attack state)
- UI components (EnemySprite, HealthBar, Buttons) read from game state
- Player input (Guard/Duck) updates player state, which affects fight system behavior
- When health reaches 0, FightScreen triggers navigation to ResultScreen

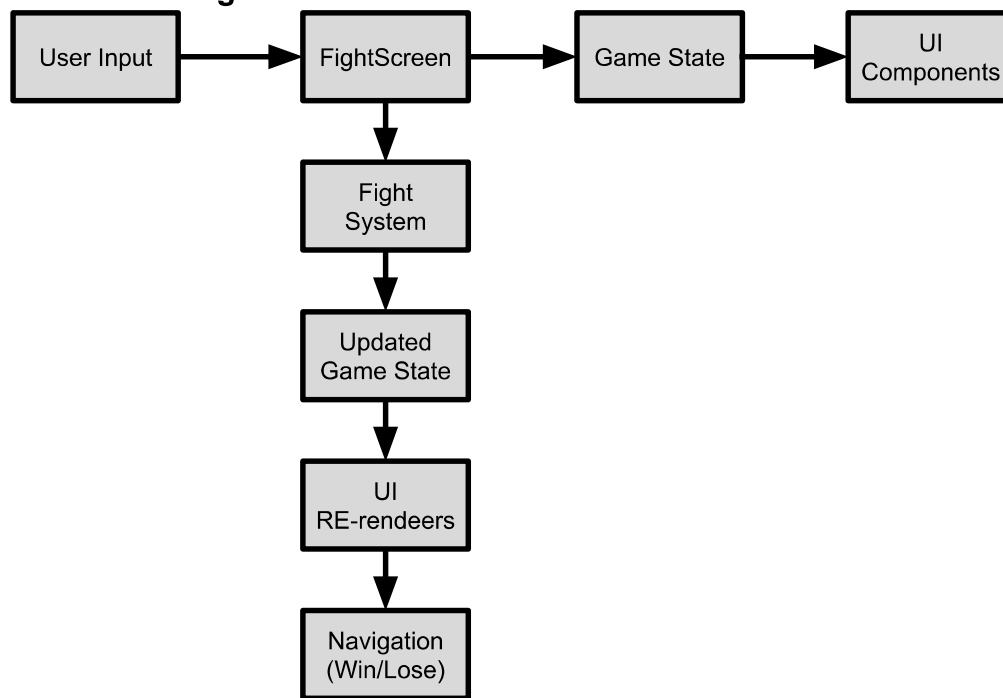
This flow ensures that UI, logic, and data remain connected but separated in responsibility.

Diagrams

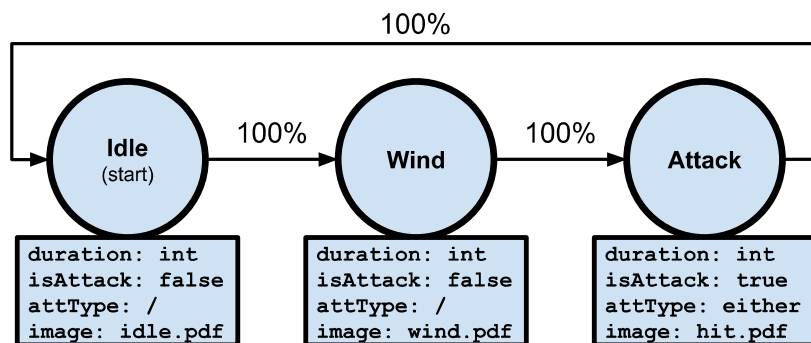
Component Hierarchy Diagram



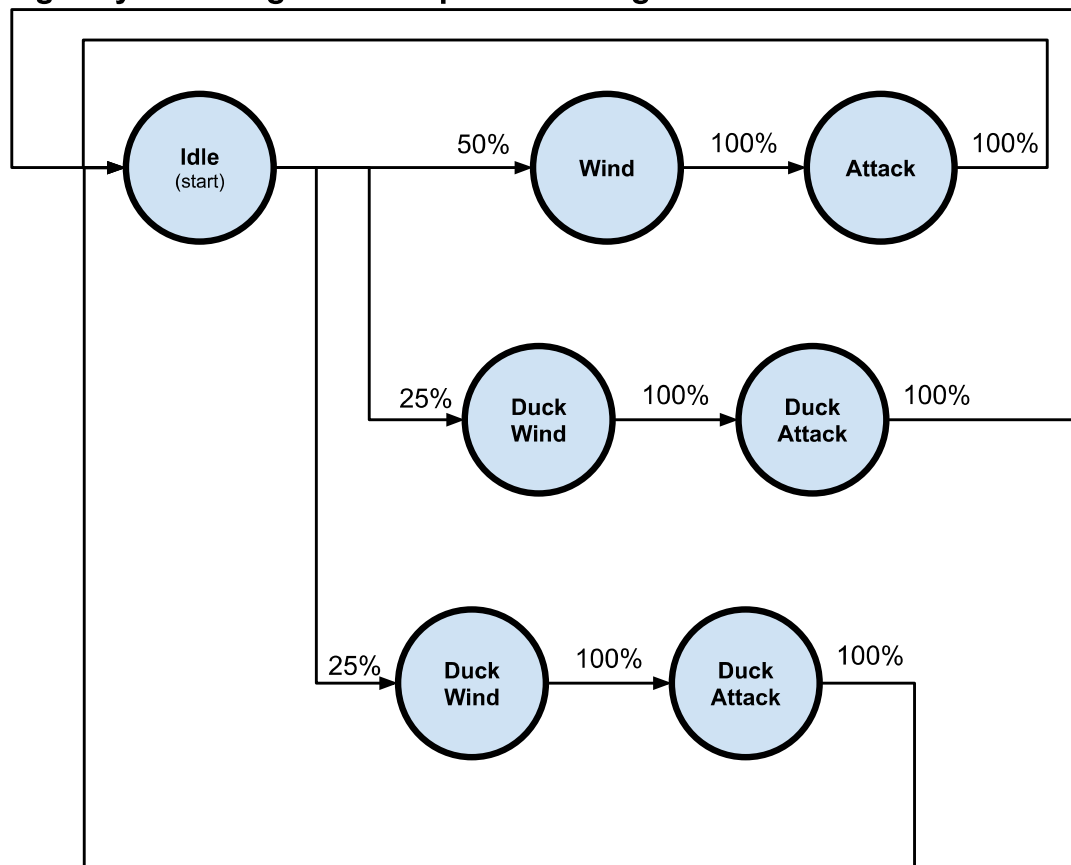
Data Flow Diagram



Fight System Diagram - Basic

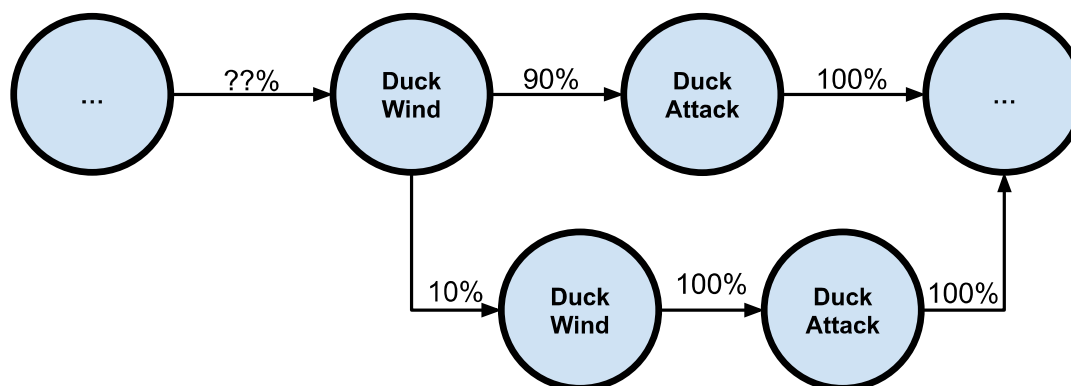


Fight System Diagram Example - All 7 Edges

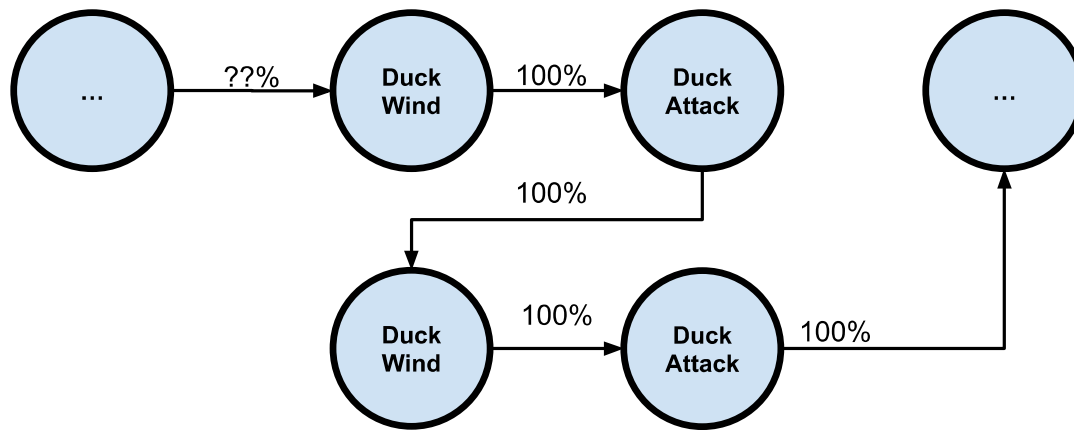


Fight System Diagram Example - More Complex Attacks

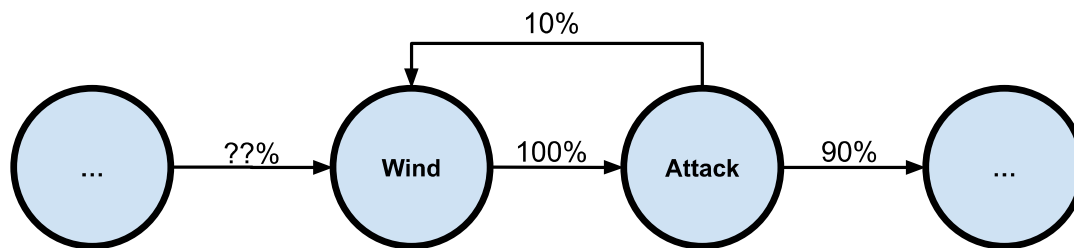
Feint:



String:



Repeat:



Shuffle:

